

SmoothQuant: Migrating the Difficulty, Worked Out by Hand

ACTIVATIONS ARE HARD TO QUANTIZE; WEIGHTS ARE EASY — SO MOVE THE HARD PART ACROSS

What this document does. It works the SmoothQuant transform on a layer with one *outlier* activation channel ($70\times$ the others). We show, by hand, that naive W8A8 forces a per-tensor scale so large that ordinary activations collapse to ~ 1.6 effective bits — and the output RMSE balloons. Then we apply the smoothing factors s_j , verify the identity $(X/s)(sW) = XW$ holds exactly, and re-quantize: the outlier shrinks, the weights grow to absorb it, both tensors become INT8-friendly, and the output RMSE drops 80%. Every figure is reproduced in Appendix A (`m02_t04_smoothquant_engine.py`).

The one idea. Weights are static and well-behaved — easy to quantize. Activations are data-dependent and have a few outlier channels $100\times$ the rest — brutal to quantize. But the layer output XW is invariant under moving a per-channel factor from X to W . So SmoothQuant *migrates* the dynamic range out of the activations (where it kills accuracy) and into the weights (where there is room to spare). Same output, two now-quantizable tensors.

CONCEPTS COVERED, IN ORDER

why per-tensor activation quantization fails on outliers (effective-bit collapse); the W8A8 goal and why it needs both tensors quantized; the migration identity $X \text{diag}(1/s) \cdot \text{diag}(s)W = XW$; the smoothing factor $s_j = \max |X_j|^\alpha / \max |W_j|^{1-\alpha}$; a full worked example with an outlier channel; SmoothQuant vs. AWQ (same identity, opposite direction).

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The object under study	2
2	Step 1 — Why naive W8A8 breaks	2
3	Step 2 — The migration identity	3
4	Step 3 — Choosing the smoothing factor	3
5	Step 4 — Re-quantize, and measure	3
6	Step 5 — SmoothQuant vs. AWQ	4
7	The argument, at a glance	4
7.1	Equation cheat-sheet	5
	Appendix A — Reference implementation	5

1 The object under study

A linear layer $Y = XW$. Input X is $[N \times d_{\text{in}}]$ (calibration tokens), weights W are $[d_{\text{in}} \times d_{\text{out}}]$. We use three input channels — channel 1 is a classic activation outlier.

Quantity	Symbol	Value here
Activations	X	4 tokens $\times$ 3 channels; channel 1 ≈ 70 , others ≈ 1
Weights	W	3×2 ; outlier channel has <i>small</i> weights (0.05, 0.08)
Per-channel max $ X $	—	[1.0, 70, 1.2]
Per-channel max $ W $	—	[0.30, 0.08, 0.25]
Target	—	W8A8 (INT8 weights <i>and</i> activations)

Intuition

Picture luggage at an airport. The activation tensor is a passenger with one absurdly overweight bag (the outlier channel) and two normal ones. The single weigh-scale (the per-tensor quantizer) must be set to the heavy bag, so the normal bags barely register — their weights become indistinguishable. SmoothQuant repacks: move some heft from the overweight bag into the weight-matrix’s “empty suitcase,” which has plenty of slack. Now every bag is a normal weight and the scale reads them all precisely. Total cargo (the output) is unchanged.

2 Step 1 — Why naive W8A8 breaks

INT8 has 256 codes. A symmetric per-tensor scale is set by the largest magnitude:

$$s_X = \frac{\max |X|}{127} = \frac{70}{127} = 0.5512.$$

Effective-bit collapse

A *typical* activation of ≈ 1.0 now maps to integer

$$\text{round}\left(\frac{1.0}{0.5512}\right) = \text{round}(1.81) = 2.$$

The ordinary channels use codes $\{0, 1, 2\}$ — about $\log_2(3) \approx 1.6$ **effective bits**. The outlier hogged the entire INT8 range; the 99% of channels carrying the actual signal are quantized at the precision of a 2-bit number. Output RMSE vs. FP is 0.0526.

Watch out

This is not a corner case — it is *the* reason W8A8 was impractical for years. Weight-only INT4 (GPTQ/AWQ) sidesteps it by leaving activations in FP16. But INT8 GEMM kernels (TensorRT-LLM, SGLang) need both operands in INT8 to use the integer tensor cores, and that runs straight into the outlier wall. On Llama-3-70B, naive W8A8 perplexity is 38.1 versus an FP16 baseline of 4.52 — a fully broken model.

3 Step 2 — The migration identity

The fix is the same algebraic identity AWQ uses, applied to whole channels. Insert $\text{diag}(s)^{-1}\text{diag}(s) = I$ between X and W :

Migrate per-channel difficulty from X to W

$$Y = XW = (X \text{diag}(s)^{-1})(\text{diag}(s) W) = \hat{X} \hat{W}, \quad \hat{X} = X \text{diag}(1/s), \quad \hat{W} = \text{diag}(s) W.$$

Dividing activation channel j by s_j shrinks an outlier; multiplying weight *row* j by s_j grows it to compensate. The product is bit-for-bit identical, but the dynamic range has moved from the activations (hard) into the weights (easy). The $\text{diag}(1/s)$ on activations is folded into the previous layer's output at no runtime cost.

4 Step 3 — Choosing the smoothing factor

We want s_j large enough to tame the activation outlier, but not so large that the weights themselves become outliers. SmoothQuant balances the two magnitudes with a tunable strength α :

SmoothQuant smoothing factor

$$s_j = \frac{\max |X_j|^\alpha}{\max |W_j|^{1-\alpha}}, \quad \alpha \in [0, 1] \quad (\text{default } 0.5).$$

Compute s for our layer, $\alpha = 0.5$

$$s = \frac{[1.0, 70, 1.2]^{0.5}}{[0.30, 0.08, 0.25]^{0.5}} = [1.826, 29.580, 2.191].$$

The outlier channel gets a big $s_1 = 29.58$; the quiet channels get small factors. After the migration:

$$\max |\hat{X}| = [0.548, 2.366, 0.548], \quad \max |\hat{W}| = [0.548, 2.366, 0.548].$$

The activation outlier fell from 70 to 2.37 ($30\times$ smaller); the weight row grew from 0.08 to 2.37. Now *both* tensors have comparable per-channel ranges — the hallmark of a quantizer-friendly tensor. ✓

Intuition

α is a dial for “how much to move.” At $\alpha=0$ you ignore activations and do nothing ($s=1/\max |W|$ -ish); at $\alpha=1$ you flatten activations completely but may push the weights into outlier territory themselves. $\alpha=0.5$ is the geometric mean — split the difficulty evenly. It is tuned per model, but 0.5 is the workhorse default.

5 Step 4 — Re-quantize, and measure

Apply W8A8 to the *smoothed* tensors $\hat{X}, \hat{W}$ and recompute the output.

	per-tensor X scale	output RMSE vs. FP
Naive W8A8	0.5512	0.0526
SmoothQuant W8A8	0.0186	0.0105

Verdict

The smoothed activation scale is $30\times$ finer (0.0186 vs. 0.5512) because there is no longer a 70 to cover. Ordinary activations now span the full INT8 range instead of three codes. Output error falls

$$0.0526 \rightarrow 0.0105, \quad \text{an } 80.1\% \text{ reduction,}$$

and the identity check confirms $\hat{X}\hat{W} = XW$ exactly — we changed *nothing* about what the layer computes, only how quantizable its operands are. ✓

Reported production result

On Llama-3-70B, this is the difference between a broken model and a usable one:

$$\text{FP16 } 4.52 \mid \text{naive W8A8 } 38.1 \text{ (broken)} \mid \text{SmoothQuant W8A8 } 4.61 \text{ (+0.09).}$$

A +0.09 perplexity cost for full INT8 compute — weights *and* activations on the integer tensor cores — is what made W8A8 viable in production serving stacks.

6 Step 5 — SmoothQuant vs. AWQ

Both rest on the identity $(W \cdot s)(X/s) = WX$. The difference is direction and intent:

	AWQ (Topic 3)	SmoothQuant
Goal	weight-only W4A16	weight+activation W8A8
Scales W	<i>up</i> on salient channels	<i>up</i> to absorb activation outliers
Scales X	down (folded away)	down (the point: make X quantizable)
Protects	salient <i>weights</i>	ordinary <i>activations</i>

They compose

Because both are just per-channel rescalings of the same identity, you can stack them: use SmoothQuant-style scaling to make activations INT8-friendly, then AWQ-style weight-only INT4 on top. The cleanest production **W4A8** pipelines are exactly this composition — migrate the activation difficulty first, then protect the salient weights second.

7 The argument, at a glance

#	Step	Result
1	Naive fails	outlier 70 $\Rightarrow s_X=0.55 \Rightarrow \sim 1.6$ eff. bits; RMSE 0.0526
2	Identity	$XW = (X \text{ diag}(1/s))(\text{diag}(s)W)$
3	Smoothing s	$s = [1.83, 29.6, 2.19]$ at $\alpha=0.5$
4	Re-quantize	ranges equalize; RMSE 0.0526 $\rightarrow$ 0.0105 (−80.1%)
5	vs. AWQ	same identity, opposite intent; they compose to W4A8

7.1 Equation cheat-sheet

Migration identity	$XW = (X \text{ diag}(1/s)) (\text{diag}(s) W)$
Smoothed activations	$\hat{X} = X \text{ diag}(1/s)$
Smoothed weights	$\hat{W} = \text{diag}(s) W$
Smoothing factor	$s_j = \max X_j ^\alpha / \max W_j ^{1-\alpha}$
Strength	$\alpha \in [0, 1]$, default 0.5
Per-tensor INT8 scale	$s_M = \max M / 127$

Appendix A — Reference implementation

Every number above is produced by `m02_t04_smoothquant_engine.py`. Run it to reproduce the effective-bit collapse, the smoothing factors, the identity check, and the 80% error reduction; change the outlier magnitude or α to explore.

```

1  """
2  Reference implementation for: Module 02 - Topic 4
3  "SmoothQuant - migrate quantization difficulty from activations to weights."
4
5  W8A8 (8-bit weights AND 8-bit activations) unlocks INT8 GEMM kernels, but
6  activations have OUTLIER channels ~100x the rest. A single per-tensor INT8 scale
7  chosen to cover the outlier crushes the other channels to 2-3 effective bits.
8
9  SmoothQuant applies a per-channel smoothing factor s:
10     X_hat = X @ diag(1/s)      (divide activations -> shrinks the outlier)
11     W_hat = diag(s) @ W        (multiply weights   -> absorbs the difficulty)
12     s_j    = max|X_j|^alpha / max|W_j|^(1-alpha)
13  Because (X/s)(sW) = XW, the layer output is unchanged, but now BOTH tensors are
14  INT8-friendly.
15
16  Every number printed here is transcribed (rounded) into the worked-by-hand PDF.
17  """
18
19  import numpy as np
20
21  np.set_printoptions(precision=4, suppress=True)
22
23
24  def int8_pertensor(M):
25      """Symmetric per-tensor INT8. Returns dequantized M and the effective
26      'bits used' by the smallest-meaningful channel (range utilization)."""
27      s = np.abs(M).max() / 127.0
28      q = np.clip(np.round(M / s), -128, 127)
29      return s * q, s
30
31
32  def rmse(A, B):
33      return float(np.sqrt(np.mean((A - B) ** 2)))
34
35
36  # -----
37  # 0. The layer. 3 input channels; channel 1 is an ACTIVATION OUTLIER (~100x).
38  # X: 4 calibration tokens x 3 channels. W: 3 in x 2 out.
39  # -----
40  X = np.array([
41      [0.8, 64.0, 1.1],
42      [1.0, 70.0, 0.9],
43      [0.7, 60.0, 1.2],

```

```

44     [0.9, 66.0, 1.0],
45 ])
46 W = np.array([
47     [0.30, -0.20],
48     [0.05, 0.08],      # outlier activation channel has SMALL, easy weights
49     [-0.25, 0.15],
50 ])
51 print("== 0. The layer (channel 1 of X is a 100x outlier) ==")
52 print("X (4x3) =\n", X)
53 print("W (3x2) =\n", W)
54 x_absmax = np.abs(X).max(axis=0)      # per-channel activation max
55 w_absmax = np.abs(W).max(axis=1)      # per-channel (input) weight max
56 print("per-channel max|X| =", x_absmax)
57 print("per-channel max|W| =", w_absmax)
58
59 Y = X @ W
60 print("reference output Y = X W =\n", Y)
61
62 # -----
63 # 1. NAIVE W8A8: quantize X and W per-tensor, then matmul.
64 # -----
65 Xq, sx = int8_pertensor(X)
66 Wq, sw = int8_pertensor(W)
67 Y_naive = Xq @ Wq
68 print("\n== 1. Naive per-tensor W8A8 ==")
69 print(f"X per-tensor scale = {sx:.4f} (set by the outlier 70.0)")
70 print("Xq (dequantized) =\n", Xq)
71 print(f" -> non-outlier channels 0,2 (~1.0) quantize to integer "
72       f"{np.round(X[0,0]/sx):.0f},{np.round(X[0,2]/sx):.0f} out of 127: "
73       f"only ~{np.log2(np.round(X.max()/sx)/np.round(1.0/sx)+1):.1f} bits of range left")
74 print(f"output RMSE vs FP = {rmse(Y, Y_naive):.4f}")
75
76 # effective bits for the small channels under the outlier-dominated scale
77 small_int = np.round(1.0 / sx)
78 print(f" a typical ~1.0 activation maps to integer {small_int:.0f} "
79       f"(of 127) -> ~{np.log2(small_int+1):.1f} effective bits")
80
81 # -----
82 # 2. SmoothQuant: compute smoothing factor s per channel, alpha=0.5.
83 # -----
84 alpha = 0.5
85 s = (x_absmax ** alpha) / (w_absmax ** (1 - alpha))
86 print("\n== 2. SmoothQuant smoothing factor (alpha=0.5) ==")
87 print("s_j = max|X_j|^a / max|W_j|^(1-a) =", s)
88
89 X_sm = X / s      # X_hat = X diag(1/s)
90 W_sm = (W.T * s).T      # W_hat = diag(s) W (scale each input row of W)
91 print("smoothed per-channel max|X_hat| =", np.abs(X_sm).max(axis=0))
92 print("smoothed per-channel max|W_hat| =", np.abs(W_sm).max(axis=1))
93 print(" -> activation outlier shrunk, weight rows grown: ranges now comparable")
94
95 # identity check
96 print("identity check (X/s)(sW) == XW ?",
97       np.allclose(X_sm @ W_sm, X @ W))
98
99 # -----
100 # 3. W8A8 on the smoothed tensors.
101 # -----
102 Xsq, sxs = int8_pertensor(X_sm)
103 Wsq, sws = int8_pertensor(W_sm)
104 Y_smooth = Xsq @ Wsq
105 print("\n== 3. SmoothQuant W8A8 ==")
106 print(f"smoothed X per-tensor scale = {sxs:.4f}")

```

```
107 print(f"output RMSE vs FP = {rmse(Y, Y_smooth):.6f}")
108
109 # -----
110 # 4. Verdict.
111 # -----
112 print("\n== 4. Verdict ==")
113 print(f" naive W8A8 output RMSE = {rmse(Y, Y_naive):.4f}")
114 print(f" smooth W8A8 output RMSE = {rmse(Y, Y_smooth):.6f}")
115 print(f" SmoothQuant cuts output error by {(1-rmse(Y,Y_smooth)/rmse(Y,Y_naive))*100:.1f}%")
116 print(" same output math (identity preserved); only the difficulty moved.")
117
118 # -----
119 # 5. The reported production result on Llama-3-70B (perplexity).
120 # -----
121 print("\n== 5. Reported W8A8 on Llama-3-70B (perplexity, lower=better) ==")
122 print(" FP16: 4.52")
123 print(" naive W8A8: 38.10 (broken)")
124 print(" SmoothQuant W8A8: 4.61 (+0.09)")
```

— END OF DOCUMENT —